Práctica de Laboratorio 9. Módulo de Exportación, Implementación de Polimorfismo en Arquitectura MVC

Objetivo de la práctica

Integrar los conceptos de Herencia y Polimorfismo dentro de una arquitectura Modelo-Vista-Controlador (MVC) existente. Los estudiantes desarrollarán un módulo que permita exportar los registros de la base de datos de "Productos" a múltiples formatos (CSV, TXT, JSON y PDF) aplicando el patrón de diseño Estrategia, logrando así un bajo acoplamiento entre la lógica de procesamiento de archivos y el Controlador.

Requisitos previos:

- Contar con la plantilla base funcional de la práctica anterior (Lab 7. Conexión a BD, CRUD básico de la tabla productos, interfaz gráfica con Tkinter estructurada en MVC).
- Tener instalada la librería fpdf en su entorno virtual para la generación de reportes (pip install fpdf).

Descripción del Requerimiento

Actualmente, el sistema permite visualizar y registrar nuevos productos. El cliente requiere un nuevo botón en la interfaz que exporte el inventario actual a un archivo físico. El usuario debe poder elegir el formato del archivo desde un menú desplegable.

Para lograr esto sin saturar el Controlador con múltiples sentencias condicionales de lógica de formato, deberán crear una jerarquía de clases orientada a objetos que se encargue exclusivamente de la exportación.

Instrucciones de Desarrollo

Paso 1: Modificación de la Vista (Capa de Presentación)

1. En su archivo correspondiente a la Vista, ubiquen el área de controles (donde se encuentra el formulario o sobre la tabla).
2. Agreguen un componente ttk.Combobox de solo lectura que contenga las siguientes opciones: "CSV", "TXT","JSON", "PDF".
3. Agreguen un botón tk.Button o ttk.Button con el texto "Exportar Reporte".
4. Vinculen este botón a un método en el Controlador que reciba como parámetro el formato seleccionado en el Combobox.

Paso 2: Construcción del Módulo de Exportación (Lógica Orientada a Objetos)

1. Crear un nuevo archivo en su proyecto llamado exportadores.py
2. Definir una clase base abstracta llamada Exportador. Esta clase debe contener un método exportar(self, datos, ruta_archivo) que lance un error si no es sobrescrito (simulando una interfaz).
3. Crean cuatro clases hijas que hereden de Exportador:
    - ExportadorCSV: Utilizará la librería nativa csv.
    - ExportadorTXT: Utilizará la función nativa open() para iterar y escribir líneas simples.
    - ExportadorJSON: Utilizará la librería nativa json.
    - ExportadorPDF: Utilizará la librería de terceros fpdf.
4. Cada clase hija debe sobrescribir el método exportar() implementando la lógica específica de su formato.

Paso 3: Integración mediante Polimorfismo en el Controlador

1. En su Controlador, definir el método que responde al evento del botón de la Vista (ej. generar_reporte()).
2. Dentro de este método, soliciten al Modelo la lista de todos los productos actuales (generalmente retornados como una lista de tuplas o diccionarios).
3. Con base en el formato seleccionado en la Vista, instancien **únicamente** la clase hija correspondiente (ej. si eligió PDF, instanciarán ExportadorPDF).
4. Invoquen el método .exportar() del objeto instanciado pasándole los datos del Modelo. Al Controlador no le debe importar qué formato se está generando; el Polimorfismo asegura que el objeto instanciado sabrá cómo comportarse.

Material Base

Para estandarizar el desarrollo de las clases hijas y evitar cuellos de botella técnicos con el manejo de librerías externas, basarse en la siguiente estructura.

Estructura base sugerida para exportadores.py:

```
import csv

import json

from fpdf import FPDF


class Exportador:

    def exportar(self, datos, ruta_archivo):

        """Método que debe ser sobrescrito por las clases hijas"""

                raise NotImplementedError("Este método debe ser implementado por la subclase.")
```

```
class ExportadorTXT(Exportador):
    def exportar(self, datos, ruta_archivo):
        with open(ruta_archivo, 'w', encoding='utf-8') as file:
            file.write("--- REPORTE DE PRODUCTOS ---\n")
            for producto in datos:
                 # Ajustar los índices según cómo su Modelo retorna los datos de la BD
                          file.write(f"ID: {producto[0]} | Nombre: {producto[1]} | Precio: ${producto[2]}\n")
        print(f"Archivo TXT generado en: {ruta_archivo}")


# TODO: Implementar ExportadorCSV (heredando de Exportador)
# TODO: Implementar ExportadorJSON (heredando de Exportador)
```

Código de guía para ExportadorPDF: La generación de PDF requiere inicializar el documento, agregar una página y definir fuentes antes de escribir. Adapten este código dentro del método exportar de su clase ExportadorPDF.

```
class ExportadorPDF(Exportador):
    def exportar(self, datos, ruta_archivo):
        pdf = FPDF()
        pdf.add_page()

        # Configuración del título
        pdf.set_font("Arial", 'B', 16)
             pdf.cell(200, 10, txt="Reporte de Inventario de Productos", ln=True, align='C')

        # Salto de línea
        pdf.ln(10)

        # Configuración del cuerpo del documento
        pdf.set_font("Arial", size=12)

        for producto in datos:
             # Reemplazar producto[1], producto[2], etc., según la estructura de sus datos
              texto_linea = f"Producto: {producto[1]} - Precio: ${producto[2]}"
```

```
            pdf.cell(200, 10, txt=texto_linea, ln=True)

    pdf.output(ruta_archivo)
    print(f"Archivo PDF generado en: {ruta_archivo}")
```

Criterios de Evaluación:

1. **Arquitectura:** Las librerías de exportación (csv, fpdf, etc.) no deben ser importadas ni utilizadas en el Controlador ni en la Vista, únicamente en el archivo de exportadores.
2. **Polimorfismo:** El Controlador debe llamar al método .exportar() de manera genérica, sin importar qué clase específica está instanciada en ese momento.
3. **Funcionalidad:** Los cuatro archivos generados deben poder abrirse correctamente en el sistema operativo mostrando los registros reales extraídos de la base de datos.